AMSS 2026 — Lecture 9: Patterns II — Integration & Critique

Traian-Florin Șerbănuță

2026

Today's Agenda

1. Frame: from selecting a pattern to verifying it
2. Seven patterns and their signature structures
3. Live opener: drive AI to "apply" a pattern
4. Reading critically: applied, or just labeled?
5. How to verify a pattern claim
6. Bridge to W10 + your project

Recap: Architect + Critic

- **Architect / director (B):** drive AI through the SDLC.
- **Critic / reviewer (A):** read AI's output and name what's wrong or missing.

In W8 you decided *whether* a pattern earns its place. Today: *was it actually applied* — or just labeled?

From Selecting to Verifying

- **W8 pinned the decision** — is a pattern warranted here?
- **Today: the verification** — AI says "I applied the Visitor pattern." Did it build the structure, or just write the name?

Today's question: **is the pattern actually there — or is the label decoration?**

Literacy Floor: F1 + F3 + F4

From W1: in the oral defense, *unaided*, you must demonstrate:

- **F1** — read & critique AI-generated artifacts on the spot.
- **F3** — articulate why you directed AI a certain way.
- **F4** — defend traceability across your project.

Today is **F1** on patterns: read a pattern claim and verify the structure is actually present. The label is a claim; the structure is the evidence.

Seven Patterns, One Question

Recognise the intent of each:

- **Adapter** — convert one interface into another.
- **Decorator** — add behaviour by wrapping, same interface.
- **Proxy** — stand in for an object, control access.
- **Composite** — treat a tree of objects uniformly.
- **Bridge** — separate an abstraction from its implementation.
- **Visitor** — add operations without changing the elements.
- **Mediator** — centralise how objects interact.

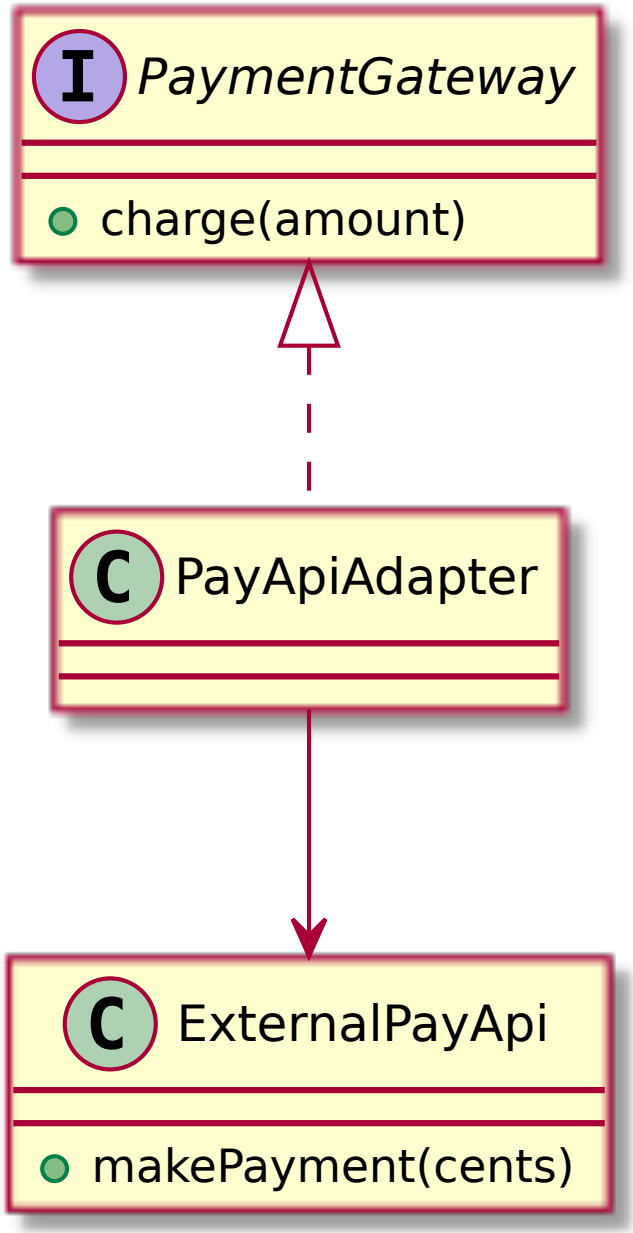
Each Pattern Has a Signature Structure

A pattern is **not a name** — it's a specific structure that delivers a specific benefit.

- No structure -> the name is empty.
- Wrong structure -> the name is a lie.

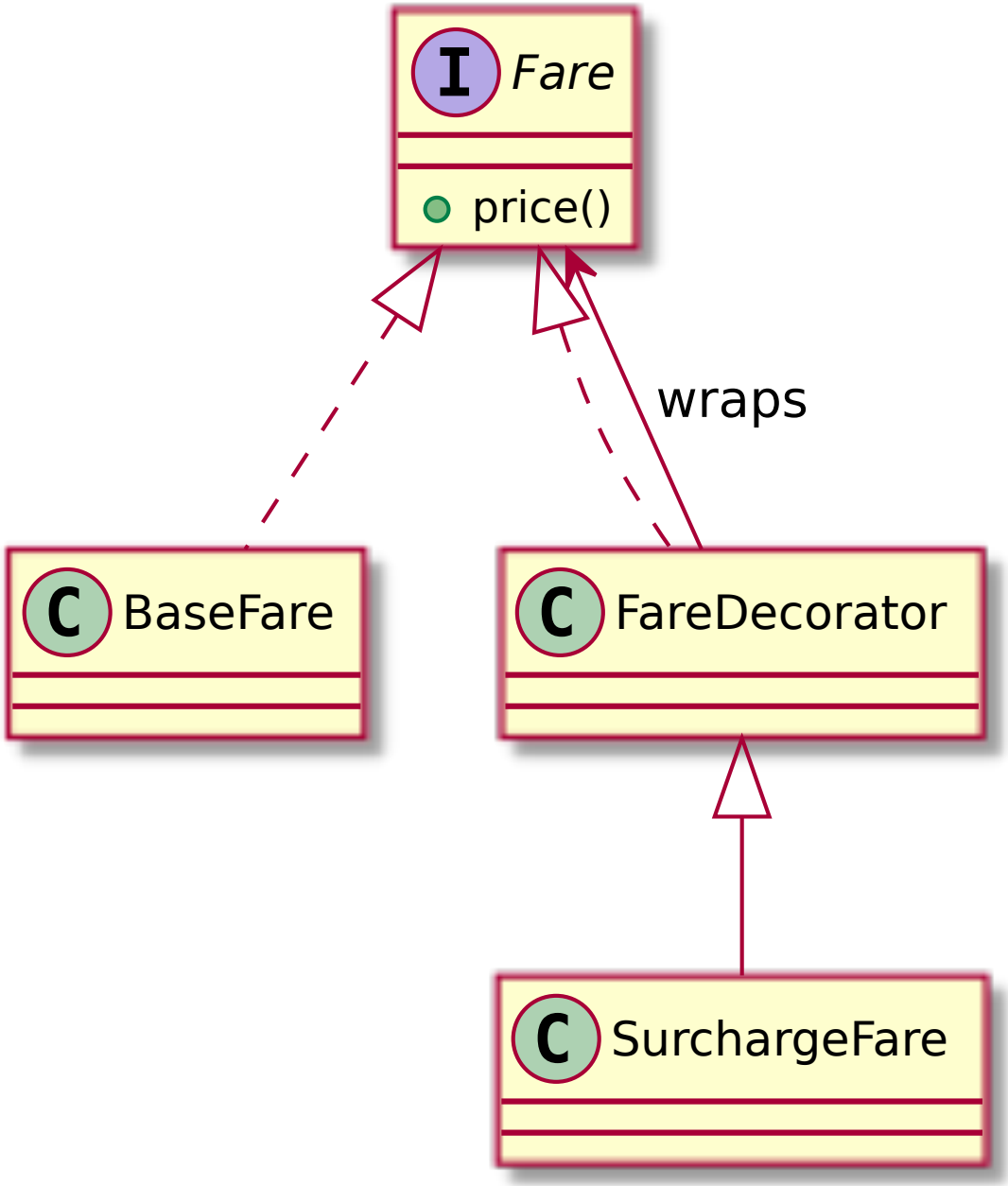
The critic verifies the structure, then the name.

Adapter — the Signature

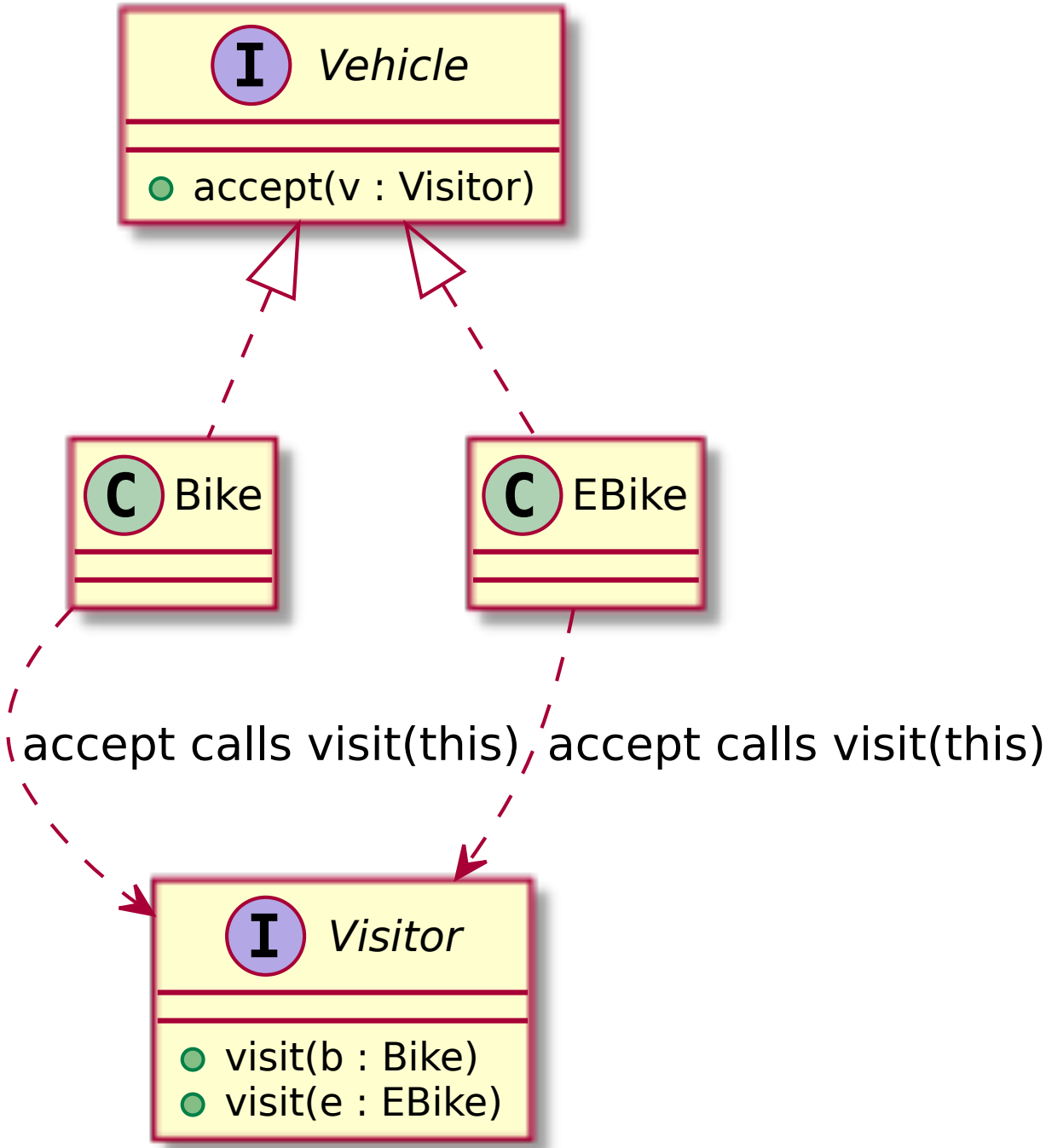


Applied — the adapter implements *our* interface and translates to

Decorator — the Signature



Visitor — the Signature



Demo: Drive AI to "Apply" a Pattern

Live: ask AI to apply the Visitor pattern. Watch whether it builds double dispatch — or writes a switch on the type and calls it Visitor.

Prompt to AI: *"Apply the Visitor pattern to compute a maintenance report across our vehicle types (Bike, EBike, Scooter) in the bike-sharing app."*

The Critic Question

When AI claims a pattern:

> *"Is the signature structure actually there — or just the name?"*

A pattern label is a claim to verify, not a fact to accept.

Defect #1: Adapter That Isn't

A "PayApiAdapter" that exposes `makePayment(cents)` directly — clients still see the external API. Or a passthrough that converts nothing.

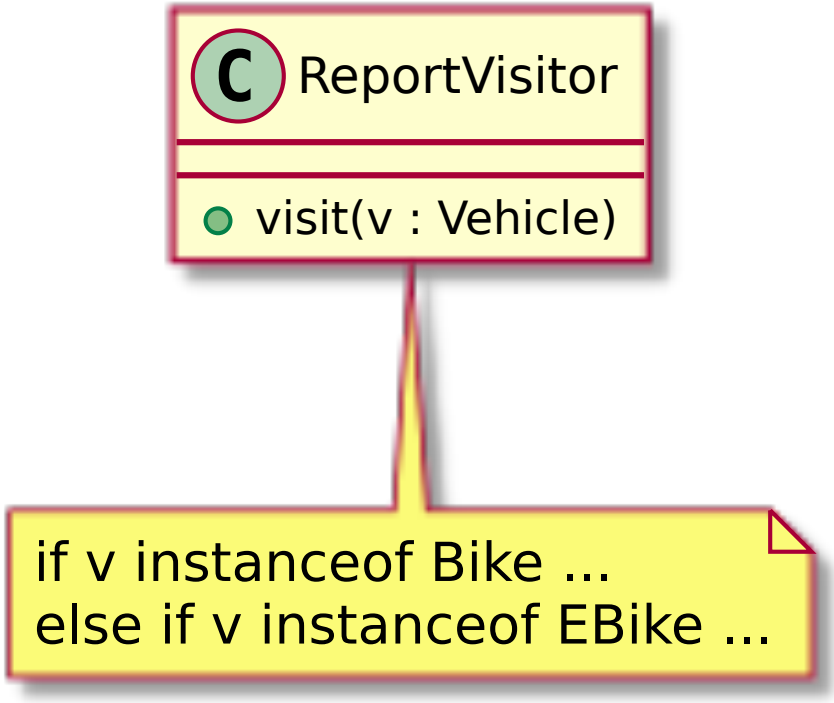
Critique: *"What interface does this adapt to? If clients still see the external API, nothing was adapted."*

Defect #2: Decorator That's Really Inheritance

A `SurchargeFare extends BaseFare` — a subclass, no wrapping, not composable. Or a "decorator" that changes the interface.

Critique: *"Can you wrap one decorator in another? If not, it's inheritance — and it can't stack."*

Defect #3: Visitor Without Double Dispatch



A single `visit(Vehicle)` with an `instanceof` cascade — labeled Visitor, no `accept`, no double dispatch.

Critique: *"Where is accept()? Where is the dispatch on type? This is the switch the pattern exists to remove."*

Defect #4: Mislabel

- A **Proxy** that *adds* behaviour — that's a Decorator.
- A **Composite** that isn't recursive — just a flat list, no tree.
- A **Bridge** with the abstraction and implementation fused.

Critique: *"Name it by its structure, not its vibe. Which pattern does this structure actually match?"*

Defect #5: Pattern Theater

Pattern names in class names and comments — `StrategyManager`, `// Observer pattern here` — with no structural reality behind them.

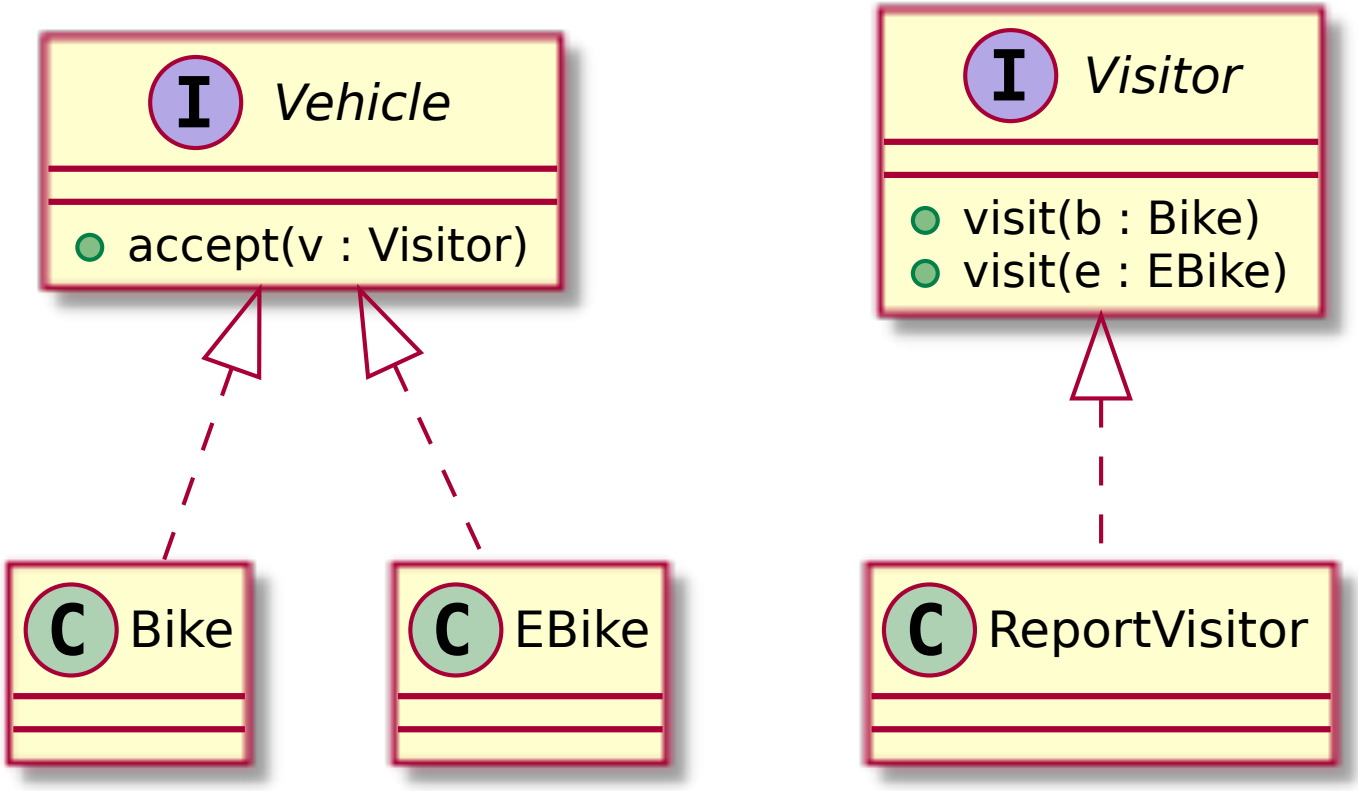
Critique: *"Strip the label. Is there a Strategy interface and interchangeable implementations — or just an if/else?"*

How to Verify a Pattern Claim

A fixed order, every time:

1. What is the pattern's **signature structure**?
2. Is that structure **actually present**?
3. Does it **deliver the benefit** the pattern exists for?
4. Is the **name correct** for the structure?

Applied, Not Labeled



The corrected Visitor: `accept` on each vehicle, `visit` per type, no `instanceof`. Structure present, benefit delivered, name correct.

The Critique Loop on Patterns

read the claim -> check the signature structure -> re-prompt “show the structure, not the label” -> re-read.

Same architect-critic loop as W2-W8 — now on a pattern claim.

The Human Decides Whether the Pattern Is Real

AI labels patterns fluently and builds them unreliably. Verifying that the structure is actually present — and delivers the benefit — is the architect-critic call.

It's what the oral defense checks, and AI can't make it for you.

This Connects to Your Project

Every pattern you claim in your design narrative, you must **defend structurally** in the oral defense.

A labeled-not-applied pattern fails F3: you can't justify a structure that isn't there.

Next Week: Cross-Layer Traceability (W10)

Patterns done — W8 (selection) and W9 (verification) cover *designing with patterns*.

W10: the full trace — requirement -> use case -> class -> state / sequence -> test — and how to keep it intact. Plus **Lab 5**, your checkpoint defense.

F1 + F3 + F4 — The Through-Line

Today you drilled:

- **F1** — verified a pattern's structure against its label, and named the fakes.
- **F3** — a claimed pattern is only defensible if its structure is real.

F4: a correctly-named, genuinely-applied pattern keeps the design narrative honest.

That's It For Today

- Next lecture (W10): cross-layer traceability.
- Next week's lab (Lab 5): project workshop + checkpoint defense.

Questions?